SGBDR

Un *système de gestion de base de données* (abr. *SGBD*) est un logiciel système servant à stocker, à manipuler ou gérer, et à partager des informations dans une base de données, en garantissant la qualité, la pérennité et la confidentialité des informations, tout en cachant la complexité des opérations.
Un SGBD (en anglais *DBMS* pour *database management system*) permet d'inscrire, de retrouver, de modifier, de trier, de transformer ou d'imprimer les informations de la base de données. Il permet d'effectuer des comptes rendus des informations enregistrées et comporte des mécanismes pour assurer la cohérence des informations, éviter des pertes d'informations dues à des pannes, assurer la confidentialité et permettre son utilisation par d'autres logiciels[1]. Selon le modèle, le SGBD peut comporter une simple interface graphique jusqu'à des langages de programmation sophistiqués.
Les systèmes de gestion de base de données sont des logiciels universels, indépendants de l'usage qui est fait des bases de données. Ils sont utilisés pour de nombreuses applications informatiques, notamment les guichets automatiques bancaires, les logiciels de réservation, les bibliothèques numériques, les logiciels d'inventaire, les progiciels de gestion intégrés ou la plupart des blogs et sites web. Il existe de nombreux systèmes de gestion de base de données. En 2008, Oracle détenait près de la moitié du marché des SGBD avec MySQL et Oracle Database. Vient ensuite IBM avec près de 20 %, laissant peu de place pour les autres acteurs.
Les SGBD sont souvent utilisés par d'autres logiciels ainsi que les administrateurs ou les développeurs. Ils peuvent être sous forme de composant logiciel, de serveur, de logiciel applicatif ou d'environnement de programmation.
En 2011, la majorité des SGBD du marché manipulent des bases de données relationnelles.

But visé

Les SGBD sont les logiciels intermédiaires entre les utilisateurs et les bases de données. Une base de données est un magasin de données composé de plusieurs fichiers manipulés exclusivement par le SGBD. Ce dernier cache la complexité de manipulation des structures de la base de données en mettant à disposition une vue synthétique du contenu[4].
L'ensemble SGBD et base de données est destiné à permettre le stockage de données d'une manière offrant de nombreux avantages par rapport à un enregistrement conventionnel dans des fichiers. Il permet d'obtenir et de modifier rapidement des données, de les partager entre plusieurs usagers. Il garantit l'absence de redondance, l'intégrité, la confidentialité et la pérennité des données tout en donnant des moyens d'éviter les éventuels conflits de modification et en cachant les détails du format de fichier des bases de données[1].
Les données sont enregistrées sous forme de suites de bits représentant des lettres, des nombres, des couleurs, des formes, ... Le SGBD comporte différents mécanismes destinés à retrouver rapidement les données et de les convertir en vue d'obtenir des informations qui aient un sens[1].

- à l'aide du SGBD plusieurs usagers et plusieurs logiciels peuvent accéder simultanément aux données. Le SGBD effectue les vérifications pour assurer qu'aucune personne non autorisée n'ait accès à des données confidentielles contenues dans la base de données, il arbitre les collisions lorsqu'il y a plusieurs modifications simultanées de la même information et comporte des mécanismes en vue d'éviter des pertes de données à la suite d'une panne[1] ;
- la redondance désigne une situation de présence de plusieurs copies de la même donnée dont la modification peut amener à des incohérences se manifestant par des copies différentes. Le SGBD vérifie - voire refuse - la présence de redondances. Le SGBD effectue également sur demande des vérifications pour assurer que les données introduites soient correctes (valeurs dans les limites admises, format correct) et que les données soient cohérentes par rapport à ce qui se trouve déjà dans la base de données[1] ;
- les données sont typiquement manipulées par un logiciel applicatif qui fait appel aux services du SGBD pour manipuler la base de données. Alors qu'un logiciel applicatif qui manipule un fichier tient compte du format de données de ce fichier, un logiciel qui manipule une base de données par l'intermédiaire d'un SGBD n'a pas connaissance du format de la base de données, les données sont présentées par le SGBD sous une forme qui cache les détails du format des fichiers dans lesquels elles sont enregistrées.

Les SGBD contemporains sont des logiciels sophistiqués nécessitant du personnel hautement qualifié, et leur utilisation entraîne souvent une augmentation substantielle des coûts liés aux licences et à la formation.

Fonctionnalités

Un SGBD permet d'enregistrer des données, puis de les rechercher, de les modifier et de créer automatiquement des comptes rendus (anglais *report*) du contenu de la base de données. Il permet de spécifier les types de données, la

structure des données contenues dans la base de données, ainsi que des règles de cohérence telles que l'absence de redondance.
Les caractéristiques des données enregistrées dans la base de données, ainsi que les relations, les règles de cohérence et les listes de contrôle d'accès sont enregistrées dans un catalogue qui se trouve à l'intérieur de la base de données et manipulé par le SGBD[5].
Les opérations de recherche et de manipulation des données, ainsi que la définition de leurs caractéristiques, des règles de cohérence et des autorisations d'accès peuvent être exprimées sous forme de *requêtes* (anglais *query*) dans un langage informatique reconnu par le SGBD. SQL est le langage informatique le plus populaire , c'est un langage normalisé de manipulation des bases de données. Il existe de nombreux autres langages comme le Databasic de Charles Bachman, Dataflex, dBase ou xBaseScript (etc.).
Les bases de données peuvent être d'une taille de plusieurs téraoctets; une taille supérieure à la place disponible dans la mémoire centrale de l'ordinateur. Les bases de données sont enregistrées sur disque dur, ces derniers ont une capacité supérieure, mais sont moins rapides, et le SGBD est équipé de mécanismes visant à accélérer les opérations[5].
Les SGBD contemporains enregistrent non seulement les données, mais également leur description, des formulaires, la définition des comptes rendus, les règles de cohérence, des procédures; ils permettent le stockage de vidéos et d'images. Le SGBD manipule les structures complexes nécessaire à la conservation de ces informations.
Les SGBD sont équipés de mécanismes qui effectuent des vérifications à l'insu de l'utilisateur, en vue d'assurer la réussite des transactions, éviter des problèmes dus aux accès concurrents et assurer la sécurité des données :

- transactions : une transaction est une opération unitaire qui transforme le contenu de la base de données d'un état A vers un état B. La transformation peut nécessiter plusieurs modifications du contenu de la base de données. Le SGBD évite qu'il existe des états intermédiaires entre A et B en garantissant que les modifications sont effectuées complètement ou pas du tout. En cas de panne survenue durant des opérations de modification de la base de données, le SGBD remet la base de données dans l'état ou elle était au début de la transaction (état A) ;
- concurrence : la base de données peut être manipulée simultanément par plusieurs personnes, et le contrôle de la concurrence vérifie que ces manipulations n'aboutissent pas à des incohérences. Par exemple dans un logiciel de réservation, le SGBD vérifie que chaque place est réservée au maximum par une personne, même si des réservations sont effectuées simultanément ;
- sécurité des données : le choix de permettre ou d'interdire l'accès à des données est donné par des listes de contrôle d'accès, et des mécanismes du SGBD empêchent des personnes non autorisées de lire ou de modifier des données pour lesquelles l'accès ne leur a pas été accordé.

Source : Wikipédia.

Cette année, nous n'étudions que des bases de données relationnelles. On utilisera donc les Systèmes de Gestion de Bases de Données Relationnelles (SGBDR).

I) LE LANGAGE SQL :

Nous avons eu l'occasion d'étudier la structure d'une base de données relationnelle, nous allons maintenant apprendre à réaliser des requêtes, c'est-à-dire que nous allons apprendre à créer une base de données, créer des attributs, ajouter de données, modifier des données et enfin, nous allons surtout apprendre à interroger une base de données afin d'obtenir des informations.
Pour réaliser toutes ces requêtes, nous allons devoir apprendre un langage de requêtes : SQL (Structured Query Language (ou langage de requête structuré)). SQL est propre aux bases de données relationnelles, les autres types de bases de données utilisent d'autres langages pour effectuer des requêtes. Il s'appuie sur les opérateurs de l'algèbre relationnelle définit par Codd, mathématicien, chercheur chez IBM. Le langage SQL est basé sur le concept de relation de la théorie des ensembles.

1°) SQL – Création d'une table :

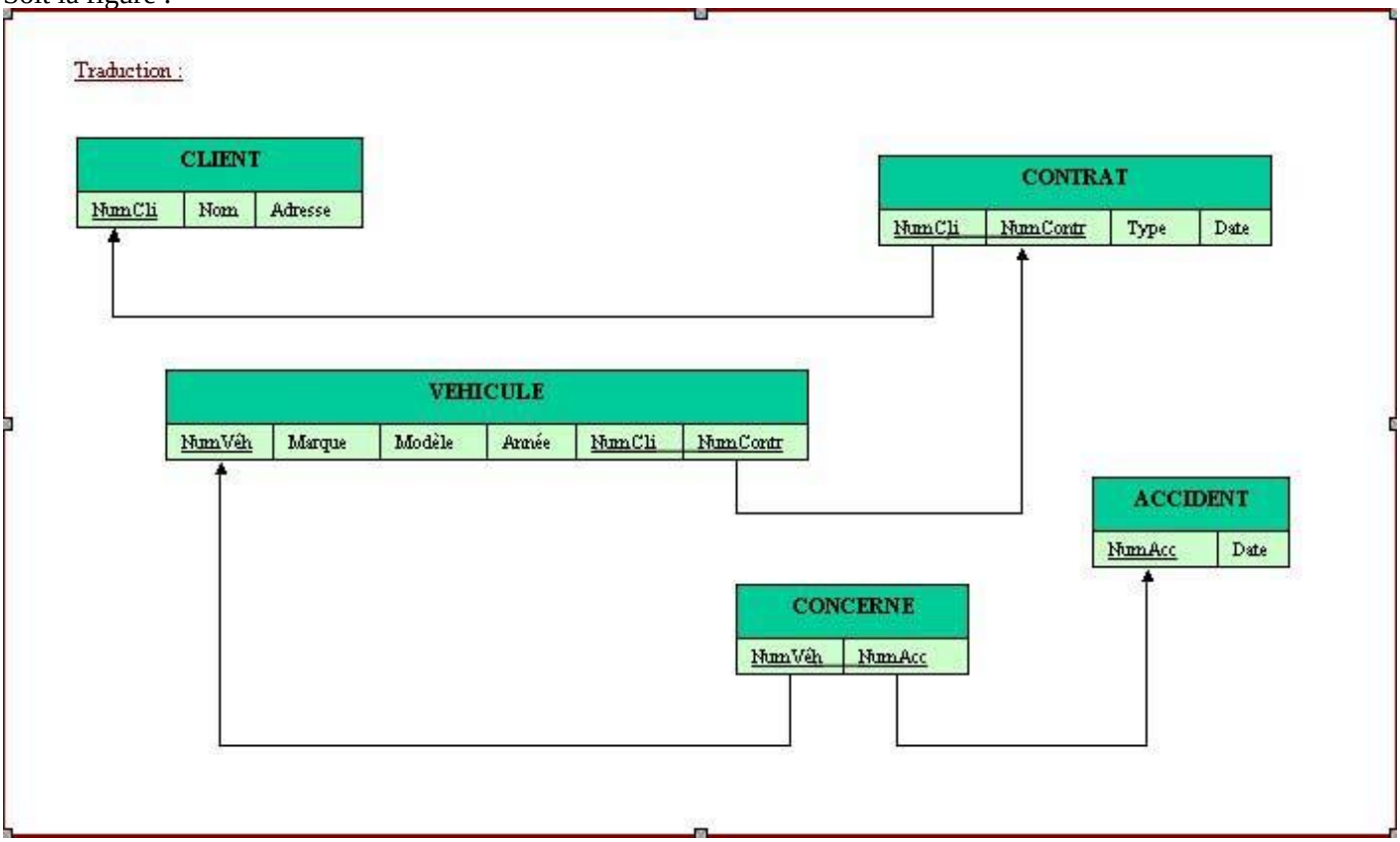
La création d'une table peut être réalisée au moyen de la commande suivante : **CREATE TABLE nom_table**

Le langage SQL dispose également des **commandes** suivantes :
ALTER TABLE : permet la modification de la structure d'une table, ajout d'une colonne, ajout d'une contrainte, modification de la définition d'une colonne, suppression d'une contrainte, etc.
DROP TABLE : permet de supprimer une table.
RENAME : permet de modifier le nom d'une table.

Les contraintes de colonnes et les contraintes de table matérialisent les différentes contraintes d'intégrité dont les SBD prend en charge la vérification systématique :

- La contrainte d'unicité (clause **UNIQUE**) : permet d'assurer qu'il n'existe pas de valeur dupliquée dans la colonne.
- La contrainte d'obligation (clause **NOT NULL**) autorise et gère une valeur particulière appelée la valeur nulle. Cette valeur nulle traduit à la fois la valeur manquante et la valeur inexistante (ex. le nom de jeune fille pour un homme). NOT NULL est toujours une contrainte colonne, et ne peut pas être une contrainte de table.
- La contrainte de clé primaire (clause **PRIMARY KEY**) : permet de choisir une colonne (ou un groupe de colonnes) unique privilégiée dans une table ;
- La contrainte d'intégrité référentielle admet deux syntaxes selon qu'elle porte sur une colonne (contrainte de colonne) ou sur plusieurs (contrainte de table). Dans le premier cas, on utilise la clause **REFERENCES**. Dans le second, on utilise l'expression **FOREIGN KEY**. La clé étrangère fait référence à la clé primaire d'une autre table en créant ainsi un lien sémantique.
- La contrainte sémantique (clause **CHECK**) : permet de spécifier les conditions logiques portant sur une ou plusieurs colonnes d'une même table.

Exemple de traduction du schéma relationnel en langage SQL :
Soit la figure :



En langage SQL :
create table CLIENT (
NumCli char(12) not null,
Nom char(38) not null,
Adresse char(60) not null,
primary key (NumCli) **)**;

create table CONTRAT (
NumCli char(12) not null,
NumContr char(38) not null,
Type decimal(4) not null,
Date date not null,
primary key (NumCli, NumCtr),
foreign key (NumCli) references CLIENT**)**;

create table VEHICULE (
NumVeh char(16) not null,
Marque char(30) not null,
Modele decimal(30) not null,
NumCli char(12) not null,
NumContr decimal(8) not null,
primary key (NumVeh),
unique (Numcli, NumContr),
foreign key (NumCli) references CONTRAT**)**;

create table ACCIDENT (
NumAcc char(10) not null,
Date date not null,
primary key (NumAcc) **)**;

create table CONCERNE (
NumVeh char(16) not null,
NumAcc char(10) not null,
primary key (NumVeh, NumAcc),
foreign key (NumVeh) references CLIENT,
foreign key (NumAcc) references CLIENT**)**;

2°) SQL – Interrogation des données :

Le langage de manipulation des données comprend quatre instructions principales :
SELECT : pour l'interrogation d'une ou plusieurs tables ;
INSERT : pour l'ajout des lignes dans une table ;
UPDATE : pour la modification des lignes ;
DELETE : pour la suppression des lignes.

Les opérations de base :

- La projection
- La sélection
- La Jointure

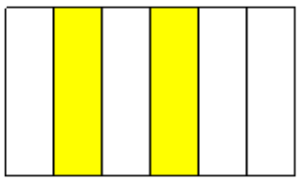
Les opérations ensemblistes :

- L'union
- L'intersection
- La différence
- Produit cartésien

a) Les opérations de base :

PROJECTION

Cet opérateur ne porte que sur 1 relation. Il permet de ne retenir que certains attributs spécifiés d'une relation. On obtient tous les n-uplets de la relation ***à l'exception des doublons***.



Opération PROJECTION

SELECT DISTINCT liste d'attributs **FROM** table **;**
SELECT liste d'attributs **FROM** table **;**
Exemples :

SELECT DISTINCT Espèce FROM Champignons ;
SELECT DISTINCT Espèce, Catégorie FROM Champignons ;

La clause DISTINCT permet d'éliminer les doublons.

SELECTION
Cet opérateur porte sur 1 relation.
Il permet de ne retenir que les n-uplets répondant à une condition exprimée à l'aide des opérateurs arithmétiques (=, >, <, >=, <=, <>) ou logiques de base (ET, OU, NON).
Tous les attributs de la relation sont conservés.
Un attribut peut ne pas avoir été renseigné pour certains n-uplets. Si une condition de sélection doit en tenir compte, on indiquera simplement : nom_attribut "non renseigné".

Opération SELECTION
SELECT * FROM table **WHERE** condition **;**
Exemple :
SELECT * FROM Champignons WHERE Catégorie="Sec" ;
La condition de sélection exprimée derrière la clause WHERE peut être spécifiée à l'aide :

- des opérateurs de comparaison : =, >, <, <=, >=, <>
- des opérateurs logiques : AND, OR, NOT
- des opérateurs : IN, BETWEEN, LIKE, IS

Autres exemples :
Soit la table ETUDIANT (N°Etudiant, Nom, Age, CodePostal, Ville)
SELECT *
FROM ETUDIANT
WHERE Age IN (19, 20, 21, 22, 23) ;
SELECT *
FROM ETUDIANT
WHERE Age BETWEEN 19 AND 23 ;
SELECT *
FROM ETUDIANT
WHERE CodePostal LIKE '42%' ;
SELECT *
FROM ETUDIANT
WHERE CodePostal LIKE '42___' ;
SELECT *
FROM ETUDIANT
WHERE Ville IS NULL ; // Etudiants pour lesquels la ville n'est pas renseignée
SELECT *
FROM ETUDIANT
WHERE Ville IS NOT NULL ; // Etudiants pour lesquels la ville est renseignée

JOINTURE (R1, R2, condition d'égalité entre attributs**)**
- Cet opérateur porte sur 2 relations qui doivent avoir au moins un attribut défini dans le même domaine (ensemble des valeurs permises pour un attribut).
- La condition de jointure peut porter sur l'égalité d'un ou de plusieurs attributs définis dans le même domaine (mais n'ayant pas forcément le même nom).
- Les n-uplets de la relation résultat sont formés par la concaténation des n-uplets des relations d'origine qui vérifient la condition de jointure.
Remarque : Des jointures plus complexes que l'équijointure peuvent être réalisées en généralisant l'usage de la condition de jointure à d'autres critères de comparaison que l'égalité (<,>, <=,>=, <>).

Opération JOINTURE (équijointure)
En SQL, il est possible d'enchaîner plusieurs jointures dans la même instruction SELECT.
En SQL de base :
SELECT * FROM table1**,** table2**,** table3**, ...**
WHERE table1.attribut1=table2.attribut1 **AND** table2.attribut2=table3.attribut2 **AND** ...**;**

Exemple :
SELECT * FROM Produit, Détail_Commande
WHERE Produit.CodePrd=Détail_Commande.CodePrd ;
ou en utilisant des alias pour les noms des tables :
SELECT * FROM Produit A, Détail_Commande B
WHERE A.CodePrd=B.CodePrd ;

La commande JOIN :
Les jointures permettent de récupérer des données dans une base de données où les tables ont des relations entre elles. Elles permettent d'exploiter les tables d'une base de données, de les lier et d'obtenir des résultats efficaces.
Il en existe plusieurs types, nous allons aborder les principales.
On va s'appuyer sur deux tables :

- la table **ville** montre les villes où vit le client correspondant à l'id_client,
- la table **email** donne les coordonnées mails des clients.

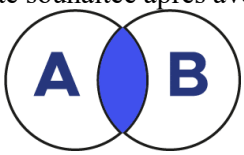
Ville

id_client	ville
1	Marseille
2	Paris
3	Lyon
7	Montpellier
8	Levallois

Email

id_client	nom	prenom	email
1	Martin	Lucas	lucas.martin@outlook.fr
2	Clavier	Paul	NULL
3	Sauron	Benoit	NULL
4	Oron	Louis	louis.oron@yahoo.fr
5	Poiret	Michel	michel76@outlook.fr

1. INNER JOIN
La jointure interne ou INNER JOIN permet de retourner les données **quand la condition est vraie dans les deux tables**.
Comme le montre le schéma, ce type de jointure va permettre de concaténer les tuples des 2 tables deux à deux si une condition est satisfaite. Cette condition peut être de tout type tant qu'elle retourne un booléen. Typiquement, cette condition sera l'égalité d'un attribut en commun. Il est nécessaire de spécifier les attributs à utiliser pour effectuer la jointure. On utilise le mot clé ON suivi de l'égalité souhaitée après avoir renseigné les tables dans l'INNER JOIN.



Intersection de deux ensembles

C'est la requête qui fait perdre le plus d'informations puisqu'elle ne sélectionne que les lignes des deux tables où il y a des informations dans les deux tables.

SELECT id_client, nom, prenom, email, ville

FROM ville

INNER JOIN email

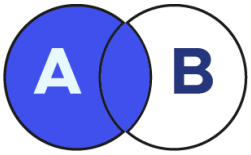
ON ville.id_client = email.id_client

id_client	nom	prenom	email	ville
1	Martin	Lucas	lucas.martin@outlook.fr	Marseille
2	Clavier	Paul	NULL	Paris
3	Sauron	Benoit	NULL	Lyon

Ici en faisant une INNER JOIN entre les deux tables on obtient seulement les id_client qui sont présents dans la table **email** et dans la table **ville**. Les id_client 4 et 5 qui sont présents dans la table **email** et non dans la table **ville** ne sont pas pris en compte. De même pour les id_client 7 et 8 qui sont dans la table **ville** et non dans la table **email**.

2. LEFT JOIN

La jointure à gauche ou LEFT JOIN est une jointure entre 2 tables qui permet de retourner tous les enregistrements de la table de gauche **même s'il n'y a pas de correspondance avec la table de droite**. S'il n'y a pas de correspondance, les valeurs manquantes sont définies à NULL. Comme pour le INNER JOIN, Il est nécessaire de spécifier les attributs à utiliser pour effectuer la jointure : On utilise le mot clé ON suivi de l'égalité souhaitée après avoir renseigné les tables dans le LEFT JOIN.



Jointure Gauche (Left Join)

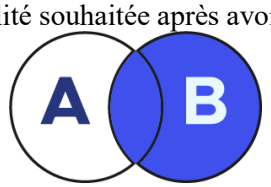
Ici on fait une requête LEFT JOIN par rapport à la table **email** donc on affiche toutes les lignes lorsque l'id_client est présent dans la table gauche donc dans la table **email**.
Ainsi les lignes correspondantes à l'id_client 7 et 8 ne sont pas présentes dans la sortie de la requête.

```
SELECT ID_CLIENT, NOM, PRENOM, VILLE
FROM EMAIL
LEFT JOIN VILLE
ON EMAIL.ID_CLIENT = VILLE.ID_CLIENT
```

id_client	nom	prenom	email	ville
1	Martin	Lucas	lucas.martin@outlook.fr	Marseille
2	Clavier	Paul	NULL	Paris
3	Sauron	Benoit	NULL	Lyon
4	Oron	Louis	louis.oron@yahoo.fr	NULL
5	Poiret	Michel	michel76@outlook.fr	NULL

3. RIGHT JOIN

La jointure à droite ou RIGHT JOIN est une jointure entre 2 tables qui permet de retourner tous les enregistrements de la table de droite **même s'il n'y a pas de correspondance avec la table de gauche**. S'il n'y a pas de correspondance, les valeurs manquantes sont définies à NULL.
Comme pour le INNER JOIN, ou le LEFT JOIN, il est nécessaire de spécifier les attributs à utiliser pour effectuer la jointure : on utilise le mot clé ON suivi de l'égalité souhaitée après avoir renseigné les tables dans le RIGHT JOIN



Jointure droite (Right Join)

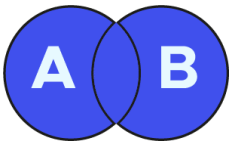
Ici on fait un RIGHT JOIN par rapport à la table **ville**, donc on obtient comme résultat toutes les lignes de la table **ville** avec les lignes correspondantes de la table **email** lorsqu'il y a correspondance, c'est pourquoi les lignes correspondant à l'id_client 4 et 5 ne sont pas présentes.

```
SELECT id_client, nom, prenom, ville
FROM email
RIGHT JOIN ville
ON email.id_client = ville.id_client
```

id_client	nom	prenom	email	ville
1	Martin	Lucas	lucas.martin@outlook.fr	Marseille
2	Clavier	Paul	NULL	Paris
3	Sauron	Benoit	NULL	Lyon
7	NULL	NULL	NULL	Montpellier
8	NULL	NULL	NULL	Levallois

4. FULL JOIN

La jointure FULL JOIN est une jointure entre 2 tables qui permet de retourner tous les enregistrements des deux tables même s'il n'y a pas de correspondance avec l'autre. S'il n'y a pas de correspondance, les valeurs manquantes sont définies à NULL. Cette jointure nécessite également l'attribut ON.



Union de deux ensembles

Ici peu importe s'il n'y a pas de lignes correspondantes dans l'une des deux tables, toutes les informations des deux tables sont présentes. C'est la jointure qui permet de garder le plus d'informations possibles.

SELECT id_client, nom, prenom, ville

FROM ville

FULL JOIN email

ON ville.id_client = email.id_client

id_client	nom	prenom	email	ville
1	Martin	Lucas	lucas.martin@outlook.fr	Marseille
2	Clavier	Paul	NULL	Paris
3	Sauron	Benoit	NULL	Lyon
4	Oron	Louis	louis.oron@yahoo.fr	NULL
5	Poiret	Michel	michel76@outlook.fr	NULL
7	NULL	NULL	NULL	Montpellier
8	NULL	NULL	NULL	Levallois

5. NATURAL JOIN

La jointure NATURAL JOIN est une jointure entre 2 tables qui permet de retourner les enregistrements des deux tables de façon « naturelle ». Il faut qu'il y ait au moins une colonne avec le même nom dans les deux tables. Elle permet de faire une jointure naturelle en retournant les lignes où il y a des paires qui correspondent aux deux tables. Avec NATURAL JOIN on ne spécifie pas le ON.

SELECT id_client, nom, prenom, ville

FROM ville

NATURAL JOIN email

id_client	nom	prenom	email	ville
1	Martin	Lucas	lucas.martin@outlook.fr	Marseille
2	Clavier	Paul	NULL	Paris
3	Sauron	Benoit	NULL	Lyon

On voit à la sortie qu'il y a bien eu une jointure entre les deux tables avec « id_client » car c'est la seule variable présente dans les deux tables.

Mais quelle est la différence entre la NATURAL JOIN et la INNER JOIN ?

La INNER JOIN permet d'éviter la répétition de lignes égales, ce que ne permet pas la NATURAL JOIN. La jointure interne renvoie une table en fonction des données spécifiées dans le ON alors que la jointure naturelle renvoie une table en fonction d'une colonne avec le même nom et le même type dans les deux tables.

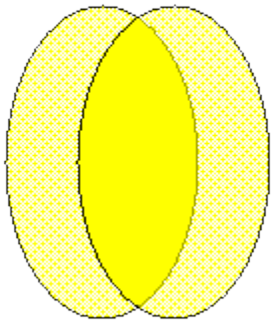
Les quatre types de jointures SQL à retenir sont :

- INNER JOIN qui permet de récupérer les informations entre deux tables lorsque la condition sur le ON est respectée sur les deux tables,
- LEFT / RIGHT JOIN qui permet de récupérer les informations entre deux tables en gardant toutes les informations de l'une des deux tables,
- FULL JOIN récupère toutes les informations des deux tables sans perte d'information (mais elle peut afficher beaucoup de NULL),
- NATURAL JOIN effectue une jointure naturelle entre les deux tables, on n'a pas besoin de spécifier le ON.

Les jointures permettent donc d'associer plusieurs tables de manière efficace. On utilisera principalement NATURAL JOIN et INNER JOIN.

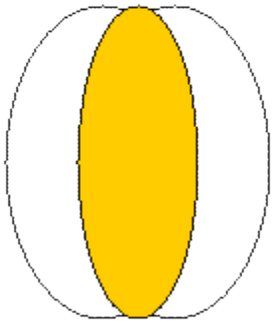
b) Les opérations ensemblistes :

UNION (R1**,** R2**)**
Cet opérateur porte sur 2 relations qui doivent avoir le même nombre d'attributs définis dans le même domaine (ensemble des valeurs permises pour un attribut). On parle de relations ayant le même schéma.
La relation résultat possède les attributs des relations d'origine et les n-uplets de chacune, avec élimination des doublons éventuels.



Opération UNION
SELECT liste d'attributs **FROM** table1
UNION
SELECT liste d'attributs **FROM** table 2 **;**
Exemple :
SELECT n°enseignant, NomEnseignant FROM E1
UNION
SELECT n°enseignant, NomEnseignant FROM E2 ;

INTERSECTION (R1**,** R2**)**
Cet opérateur porte sur deux relations de même schéma.
La relation résultat possède les attributs des relations d'origine et les n-uplets communs à chacune.



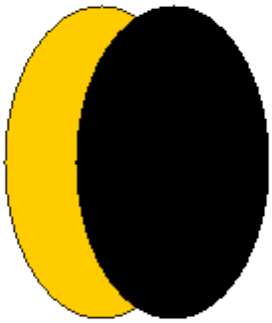
Exemple :
E1 : Enseignants élus au CA
E2 : Enseignants représentants syndicaux

On désire connaître les enseignants du CA qui sont des représentants syndicaux.
R2 = INTERSECTION (E1, E2)

Opération INTERSECTION
En SQL de base :
SELECT attribut1, attribut2, ... **FROM** table1
WHERE attribut1 **IN (SELECT** attribut1 **FROM** table2**) ;**

ou avec SQL2 :
SELECT attribut1**,** attribut2**,** ... **FROM** table1
INTERSECT
SELECT attribut1**,** attribut2**,** ... **FROM** table2 **;**
Exemple :
SELECT n°enseignant, NomEnseignant FROM E1
WHERE n°enseignant IN (SELECT n°enseignant FROM E2) ;
ou
SELECT n°enseignant, NomEnseignant FROM E1
INTERSECT
SELECT n°enseignant, NomEnseignant FROM E2 ;

DIFFERENCE (R1**,** R2**)**
Cet opérateur porte sur deux relations de même schéma.
La relation résultat possède les attributs des relations d'origine et les n-uplets de la première relation qui n'appartiennent pas à la deuxième.
Attention ! DIFFERENCE (R1, R2) ne donne pas le même résultat que DIFFERENCE (R2, R1)



Exemple : On désire obtenir la liste des enseignants du CA qui ne sont pas des représentants syndicaux.
R3 = DIFFERENCE (E1, E2)

Opération DIFFERENCE
En SQL de base :
SELECT attribut1**,** attribut2**,** ... **FROM** table1
WHERE attribut1 **NOT IN (SELECT** attribut1 **FROM** table2**) ;**
SELECT table1.attribut1, table1.attribut2,...
FROM table1 **LEFT OUTER JOIN** table2 **ON** table1.attribut1 = table2.attribut1
WHERE table2.attribut1 **IS NULL ;**
Exemple :
SELECT n°enseignant, NomEnseignant FROM E1
WHERE n°enseignant NOT IN (SELECT n°enseignant FROM E2) ;
ou
SELECT n°enseignant, NomEnseignant FROM E1
EXCEPT
SELECT n°enseignant, NomEnseignant FROM E2 ;
ou encore
SELECT E1.n°enseignant, E1.NomEnseignant
FROM E1 LEFT OUTER JOIN E2 ON E1.n°enseignant = E2.n°enseignant
WHERE E2.n°enseignant IS NULL ;
voici le résultat renvoyé par la jointure externe gauche entre E1 et E2 :

PRODUIT (R1**,** R2**)**
- Cet opérateur porte sur deux relations.
- La relation résultat possède les attributs de chacune des relations d'origine et ses n-uplets sont formés par la concaténation de chaque n-uplet de la première relation avec l'ensemble des n-uplets de la deuxième.
Exemple :
Etudiants
Epreuves

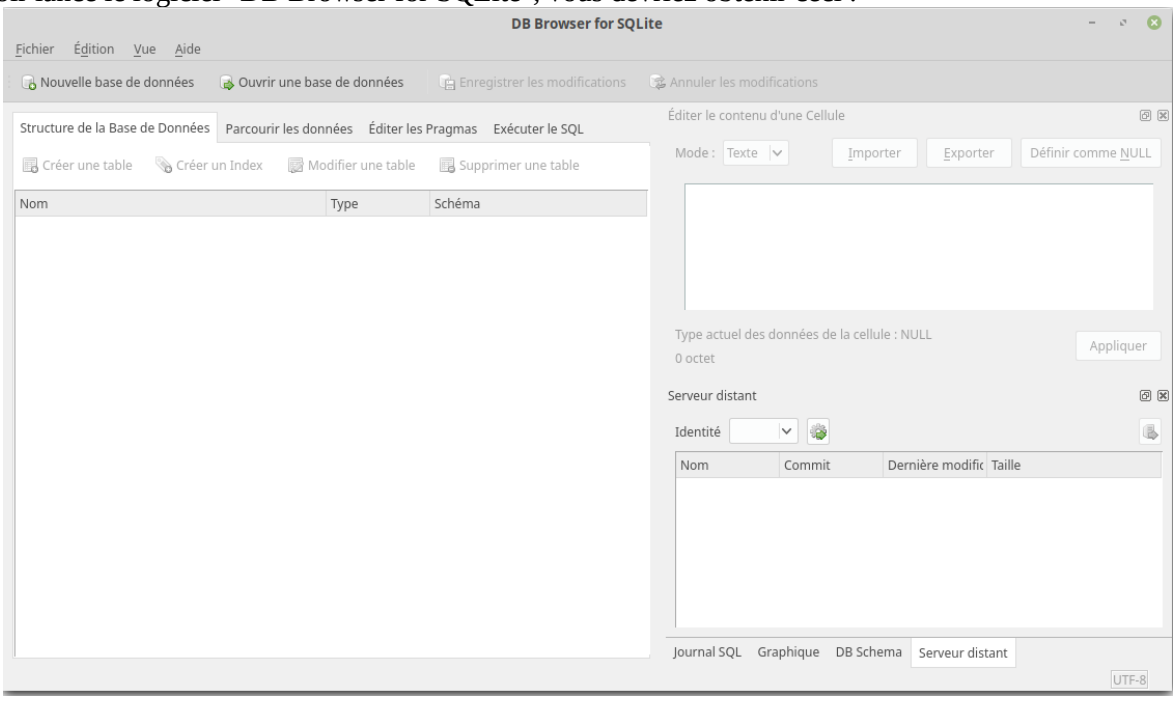
Examen = PRODUIT (Etudiants, Epreuves)

Opération PRODUIT CARTESIEN
SELECT * FROM table1**,** table2 **;**
Exemple :
SELECT * FROM Etudiants, Epreuves ;

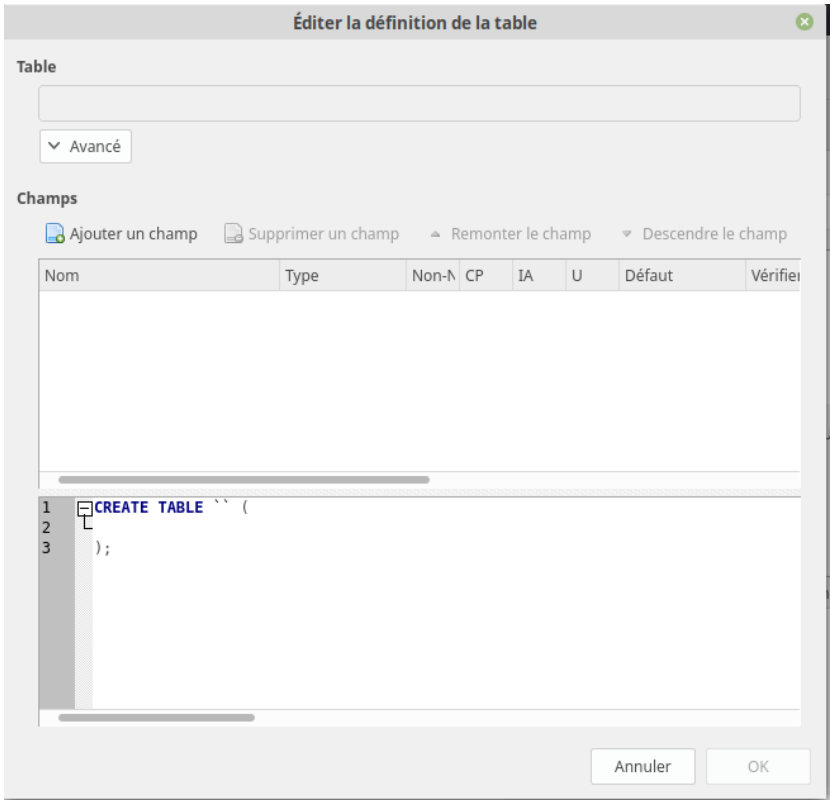
II) DB BROWSER FOR SQLITE :

Pour créer une base de données et effectuer des requêtes sur cette dernière, nous allons utiliser le logiciel "DB Browser for SQLite" : https://sqlitebrowser.org/.
SQLite est un système de gestion de base de données relationnelle très répandu. Noter qu'il existe d'autres systèmes de gestion de base de données relationnelle comme MySQL ou PostgreSQL. Dans tous les cas, le langage de requête utilisé est le SQL (même si parfois on peut noter quelques petites différences). Ce qui sera vu ici avec SQLite pourra, à quelques petites modifications près, être utilisé avec, par exemple, MySQL.
Nous allons commencer par créer notre base de données :

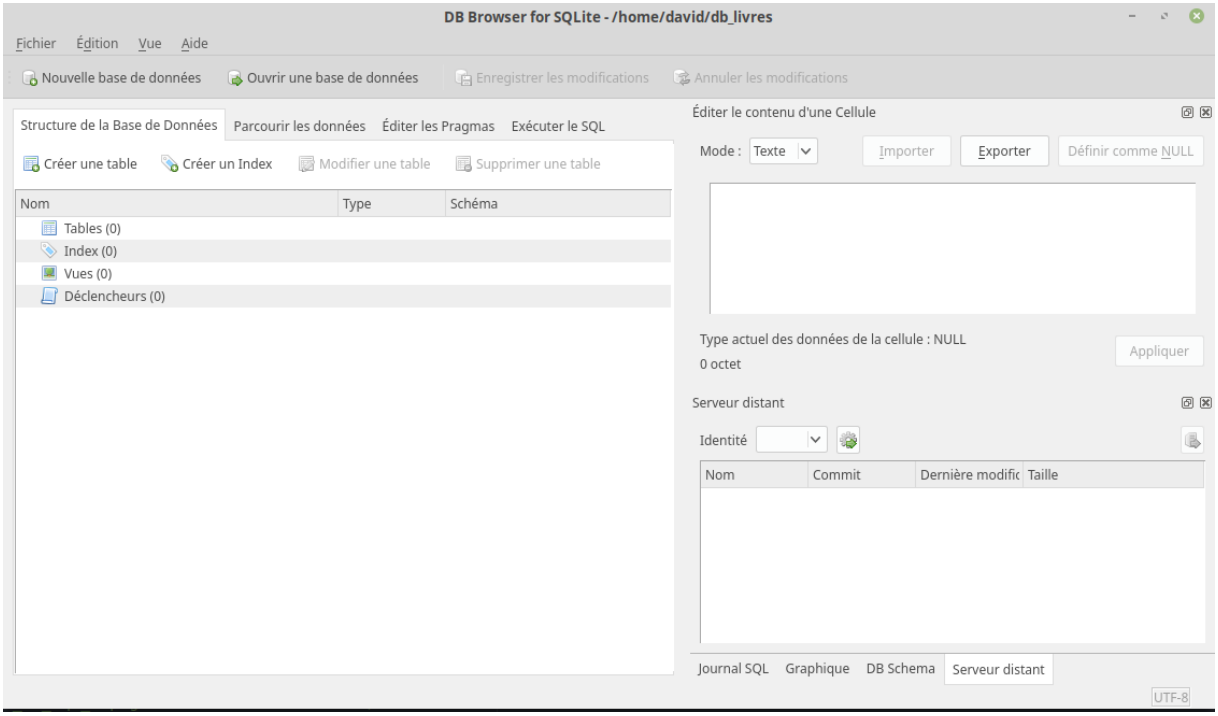
À faire vous-même 1
Après avoir lancé le logiciel "DB Browser for SQLite", vous devriez obtenir ceci :



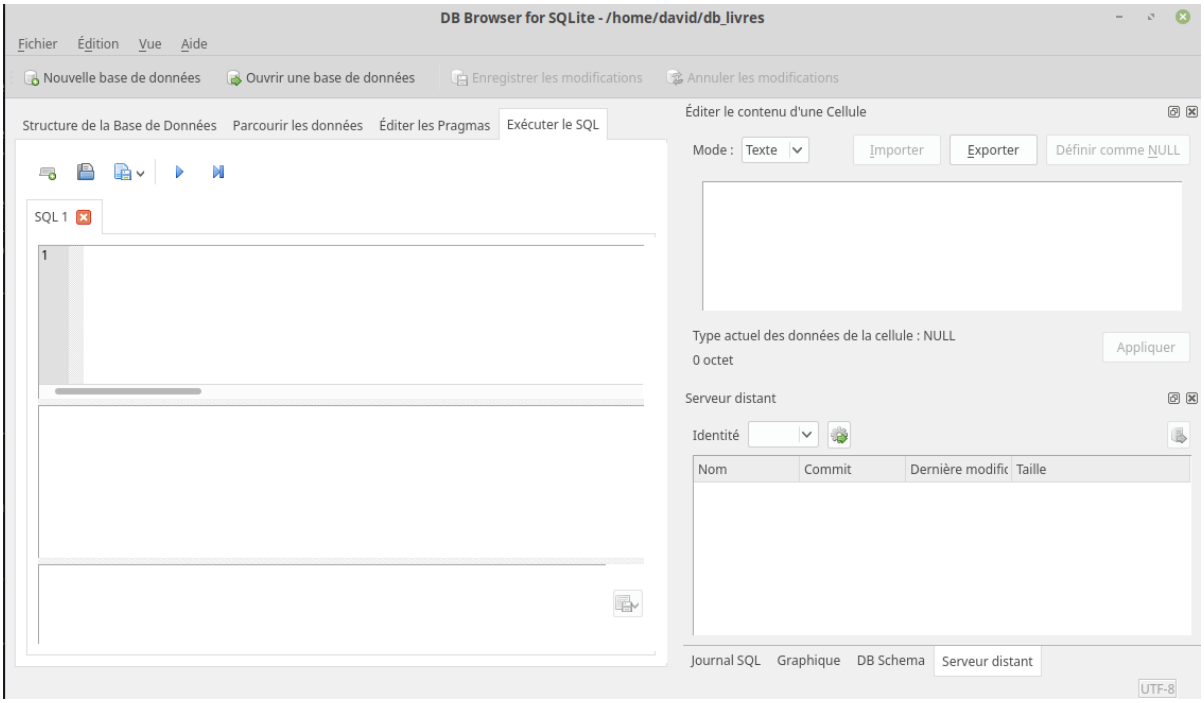
Cliquez sur Nouvelle base de données. Après avoir choisi un nom (par exemple "db_livres.db"), et un endroit de stockage pour votre base de données, vous devriez avoir la fenêtre suivante :



Cliquez alors sur Annuler.
Notre base de données a été créée :



Mais pour l'instant elle ne contient aucune table (aucune relation), pour créer une table, cliquez sur l'onglet "Exécuter le SQL". On obtient alors :



Copiez-collez le texte ci-dessous dans la fenêtre "SQL 1"

```
CREATE TABLE LIVRES
(id INT, titre TEXT, auteur TEXT, ann_publi INT, note INT);
```

Cliquez ensuite sur le petit triangle situé au-dessus de la fenêtre SQL 1 (ou appuyez sur F5), vous devriez avoir ceci :

Comme indiqué dans la fenêtre, "Requête exécutée avec succès" !
Vous venez de créer votre première table.

Revenons sur cette première requête :
Le *CREATE TABLE LIVRES* ne devrait pas vous poser de problème : nous créons une nouvelle table nommée "LIVRES".
La suite est à peine plus complexe :
nous créons ensuite les attributs :

- id
- titre
- auteur
- ann_pulbi

- note

Comme vous l'avez sans doute remarqué, nous avons pour chaque attribut précisé son domaine : id : entier (INT), titre : chaîne de caractères (TEXT), auteur : chaîne de caractères, ann_publi : entier et note : entier

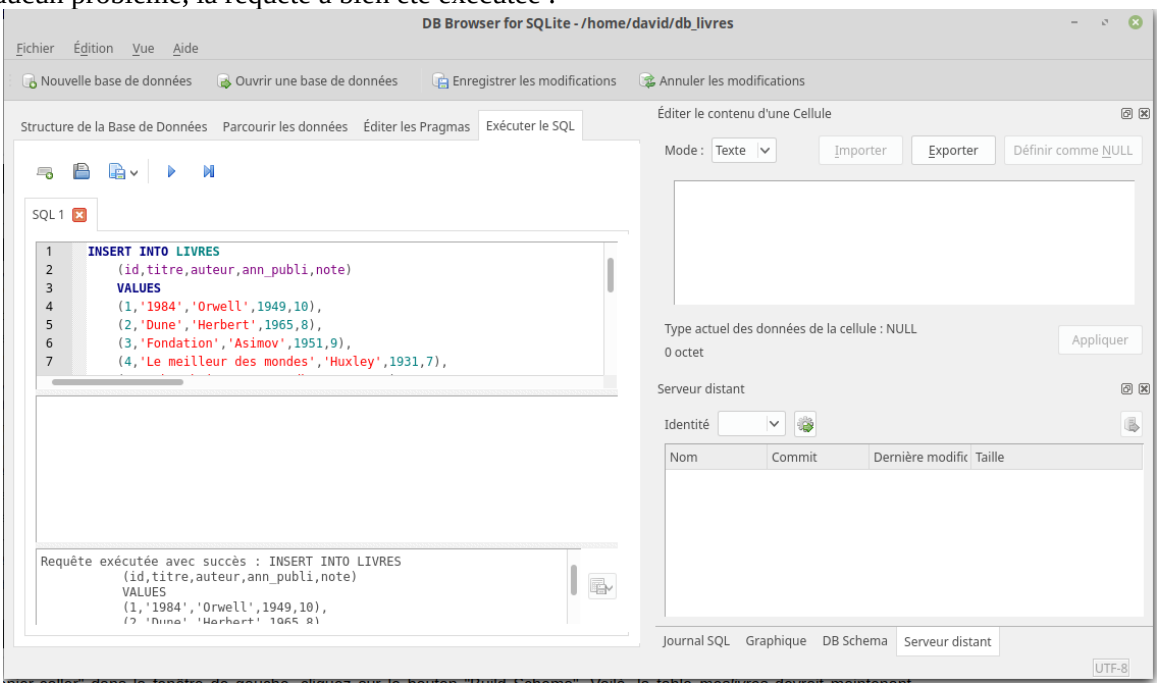
Nous allons maintenant ajouter des données :

À faire vous-même 2

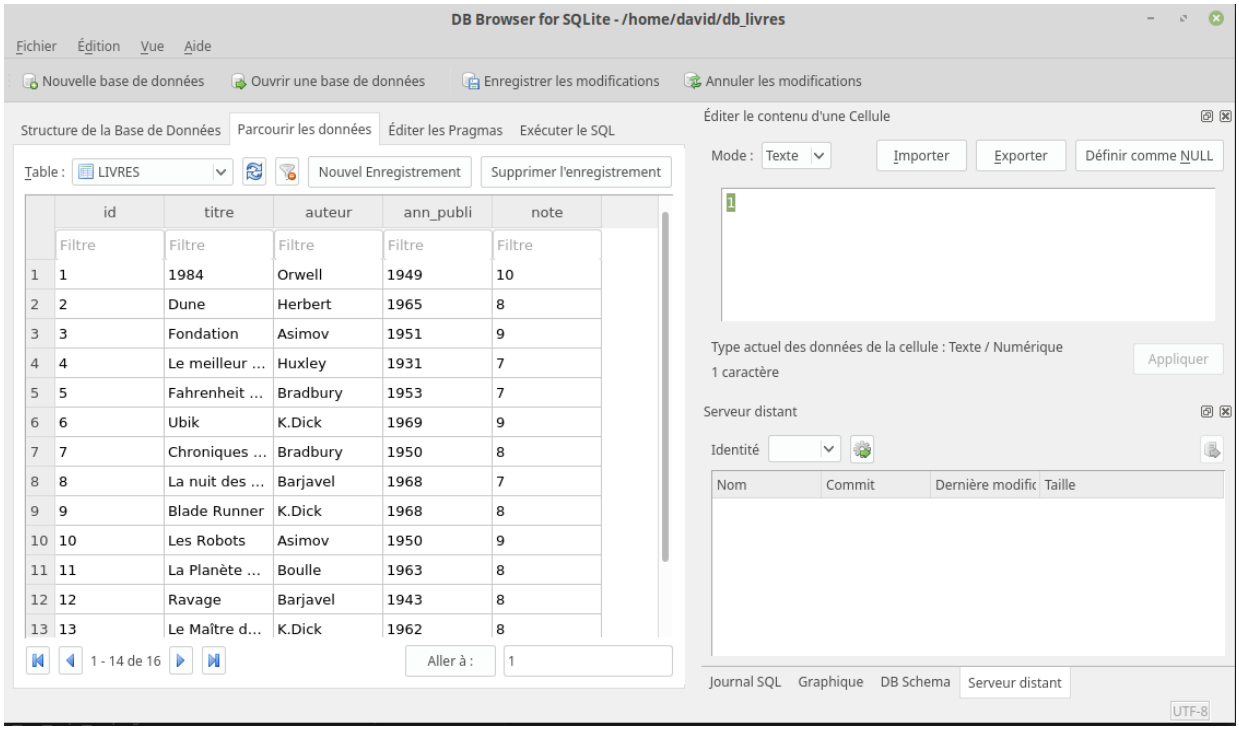
Toujours dans l'onglet "Exécuter le SQL", après avoir effacé la fenêtre SQL 1, copiez-collez dans cette même fenêtre la requête ci-dessous :

```
INSERT INTO LIVRES
(id,titre,auteur,ann_publi,note)
VALUES
(1,'1984','Orwell',1949,10),
(2,'Dune','Herbert',1965,8),
(3,'Fondation','Asimov',1951,9),
(4,'Le meilleur des mondes','Huxley',1931,7),
(5,'Fahrenheit 451','Bradbury',1953,7),
(6,'Ubik','K.Dick',1969,9),
(7,'Chroniques martiennes','Bradbury',1950,8),
(8,'La nuit des temps','Barjavel',1968,7),
(9,'Blade Runner','K.Dick',1968,8),
(10,'Les Robots','Asimov',1950,9),
(11,'La Planète des singes','Boulle',1963,8),
(12,'Ravage','Barjavel',1943,8),
(13,'Le Maître du Haut Château','K.Dick',1962,8),
(14,'Le monde des Ā','Van Vogt',1945,7),
(15,'La Fin de l'éternité','Asimov',1955,8),
(16,'De la Terre à la Lune','Verne',1865,10);
```

Ici aussi, aucun problème, la requête a bien été exécutée :



La table LIVRES contient bien les données souhaitées (onglet "Parcourir les données") :



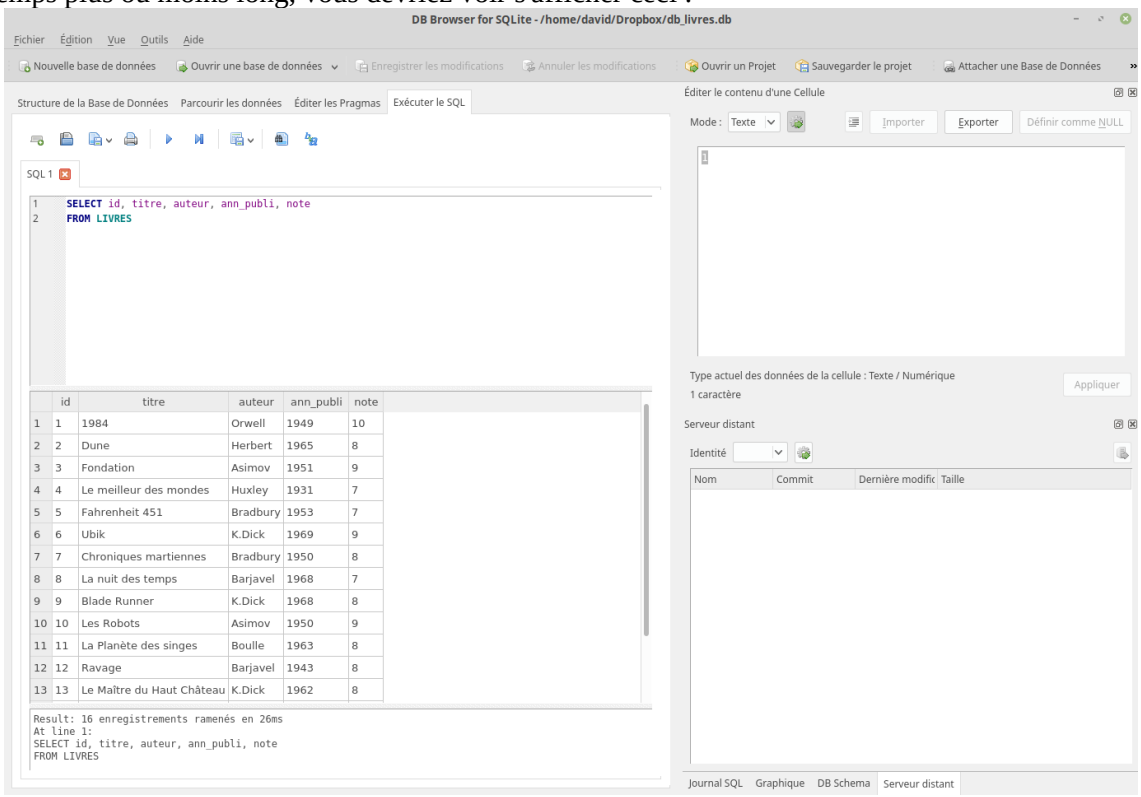
Nous allons apprendre à effectuer des requêtes d'interrogation sur la base de données que nous venons de créer. Toutes les requêtes se feront dans la fenêtre SQL 1 de l'onglet "Exécuter le SQL"

À faire vous-même 3

Saisissez la requête SQL suivante :

```
SELECT id, titre, auteur, ann_publi, note
FROM LIVRES
```

Puis appuyez sur le triangle (ou la touche F5).
Après un temps plus ou moins long, vous devriez voir s'afficher ceci :



Comme vous pouvez le constater, notre requête SQL a permis d'afficher tous les livres. Nous avons ici 2 mots clés du langage SQL *SELECT* qui permet de sélectionner les attributs qui devront être "affichés" (je mets "affichés" entre

guillemets, car le but d'une requête sql n'est pas forcément d'afficher les données) et *FROM* qui indique la table qui doit être utilisée.
Il est évidemment possible d'afficher seulement certains attributs (ou même un seul) :

À faire vous-même 4
Saisissez la requête SQL suivante :

```
SELECT titre, auteur
FROM LIVRES
```

Vérifiez que vous obtenez bien uniquement les titres et les auteurs des livres

À faire vous-même 5
Écrivez et testez une requête permettant d'obtenir uniquement les titres des livres.

N.B. Si vous désirez sélectionner tous les attributs, vous pouvez écrire :

```
SELECT *
FROM LIVRES
```

à la place de :

```
SELECT id, titre, auteur, ann_publi, note
FROM meslivres
```

Pour l'instant nos requêtes affichent tous les livres, il est possible d'utiliser la clause *WHERE* afin d'imposer une (ou des) condition(s) permettant de sélectionner uniquement certaines lignes.
La condition doit suivre le mot-clé *WHERE* :

À faire vous-même 6
Saisissez et testez la requête SQL suivante :

```
SELECT titre, ann_publi
FROM LIVRES
WHERE auteur='Asimov'
```

Vérifiez que vous obtenez bien uniquement les livres écrits par Isaac Asimov.

À faire vous-même 7
Écrivez et testez une requête permettant d'obtenir uniquement les titres des livres écrits par Philip K.Dick.

Il est possible de combiner les conditions à l'aide d'un *OR* ou d'un *AND*

À faire vous-même 8
Saisissez et testez la requête SQL suivante :

```
SELECT titre, ann_publi
FROM LIVRES
WHERE auteur='Asimov' AND ann_publi>1953
```

Vérifiez que nous obtenons bien le livre écrit par Asimov publié après 1953 (comme vous l'avez sans doute remarqué, il est possible d'utiliser les opérateurs d'inégalités).

À faire vous-même 9
D'après vous, quel est le résultat de cette requête :

```
SELECT titre
FROM LIVRES
WHERE auteur='K.Dick' OR note>=8
```

À faire vous-même 10
Écrire une requête permettant d'obtenir les titres des livres publiés après 1945 qui ont une note supérieure ou égale à 9.

Il est aussi possible de rajouter la clause SQL *ORDER BY* afin d'obtenir les résultats classés dans un ordre précis.

À faire vous-même 11
Saisissez et testez la requête SQL suivante :

```
SELECT titre
FROM LIVRES
WHERE auteur='K.Dick' ORDER BY ann_publi
```

Nous obtenons les livres de K.Dick classés du plus ancien ou plus récent.

Il est possible d'obtenir un classement en sens inverse à l'aide de la clause DESC

À faire vous-même 12
Saisissez et testez la requête SQL suivante :

```
SELECT titre
FROM LIVRES
WHERE auteur='K.Dick' ORDER BY ann_publi DESC
```

Nous obtenons les livres de K.Dick classés du plus récent au plus ancien.

À faire vous-même 13
Que se passe-t-il quand la clause ORDER BY porte sur un attribut de type TEXT ?

Vous pouvez constater qu'une requête du type :

```
SELECT auteur
FROM LIVRES
```

affiche plusieurs fois certains auteurs (les auteurs qui ont écrit plusieurs livres présents dans la base de données)
Il est possible d'éviter les doublons grâce à la clause DISTINCT

À faire vous-même 14
Saisissez et testez la requête SQL suivante :

```
SELECT DISTINCT auteur
FROM LIVRES
```

Nous avons vu précédemment qu'une base de données peut contenir plusieurs relations (plusieurs tables).

À faire vous-même 15
Créez une nouvelle base de données que vous nommerez par exemple db_livres_auteurs.db

À faire vous-même 16
Créez une table AUTEURS à l'aide de la requête SQL suivante :

```
CREATE TABLE AUTEURS
(id INT, nom TEXT, prenom TEXT, ann_naissance INT, langue_ecriture TEXT);
```

À faire vous-même 17
Créez une table LIVRES à l'aide de la requête SQL suivante :

```
CREATE TABLE LIVRES
(id INT, titre TEXT, id_auteur INT, ann_publi INT, note INT);
```

À faire vous-même 18
Ajoutez des données à la table AUTEURS à l'aide de la requête SQL suivante :

```
INSERT INTO AUTEURS
(id,nom,prenom,ann_naissance,langue_ecriture)
VALUES
(1,'Orwell','George',1903,'anglais'),
(2,'Herbert','Frank',1920,'anglais'),
(3,'Asimov','Isaac',1920,'anglais'),
(4,'Huxley','Aldous',1894,'anglais'),
(5,'Bradbury','Ray',1920,'anglais'),
```

```
(6,'K.Dick','Philip',1928,'anglais'),
(7,'Barjavel','René',1911,'français'),
(8,'Boulle','Pierre',1912,'français'),
(9,'Van Vogt','Alfred Elton',1912,'anglais'),
(10,'Verne','Jules',1828,'français');
```

À faire vous-même 19

Ajoutez des données à la table LIVRES à l'aide de la requête SQL suivante :

```
INSERT INTO LIVRES
(id,titre,id_auteur,ann_publi,note)
VALUES
(1,'1984',1,1949,10),
(2,'Dune',2,1965,8),
(3,'Fondation',3,1951,9),
(4,'Le meilleur des mondes',4,1931,7),
(5,'Fahrenheit 451',5,1953,7),
(6,'Ubik',6,1969,9),
(7,'Chroniques martiennes',5,1950,8),
(8,'La nuit des temps',7,1968,7),
(9,'Blade Runner',6,1968,8),
(10,'Les Robots',3,1950,9),
(11,'La Planète des singes',8,1963,8),
(12,'Ravage',7,1943,8),
(13,'Le Maître du Haut Château',6,1962,8),
(14,'Le monde des Ā',9,1945,7),
(15,'La Fin de l'éternité',3,1955,8),
(16,'De la Terre à la Lune',10,1865,10);
```

Nous avons 2 tables, grâce aux jointures nous allons pouvoir associer ces 2 tables dans une même requête.
En général, les jointures consistent à associer des lignes de 2 tables. Elles permettent d'établir un lien entre 2 tables.
Qui dit lien entre 2 tables dit souvent clef étrangère et clef primaire.

À faire vous-même 20

Saisissez et testez la requête SQL suivante :

```
SELECT *
FROM LIVRES
INNER JOIN AUTEURS ON LIVRES.id_auteur = AUTEURS.id
```

Le "FROM LIVRES INNER JOIN AUTEURS" permet de créer une jointure entre les tables LIVRES et AUTEURS ("rassembler" les tables LIVRES et AUTEURS en une seule grande table). Le "ON LIVRES.id_auteur = AUTEURS.id" signifie qu'une ligne quelconque A de la table LIVRES devra être fusionnée avec la ligne B de la table AUTEURS à condition que l'attribut id de la ligne A soit égal à l'attribut id_auteur de la ligne B.
Par exemple, la ligne 1 (id=1) de la table LIVRES (que l'on nommera dans la suite ligne A) sera fusionnée avec la ligne 1 (id=1) de la table AUTEURS (que l'on nommera dans la suite B) car l'attribut id_auteur de la ligne A est égal à 1 et l'attribut id de la ligne B est aussi égal à 1.
Autre exemple, la ligne 1 (id=1) de la table LIVRES (que l'on nommera dans la suite ligne A) ne sera pas fusionnée avec la ligne 2 (id=2) de la table AUTEURS (que l'on nommera dans la suite B') car l'attribut id_auteur de la ligne A est égal à 1 alors que l'attribut id de la ligne B' est égal à 2.
Cette notion de jointure n'est pas évidente, prenez votre temps pour bien réfléchir et surtout n'hésitez pas à poser des questions.

À faire vous-même 21

Saisissez et testez la requête SQL suivante :

```
SELECT *
FROM AUTEURS
INNER JOIN LIVRES ON LIVRES.id_auteur = AUTEURS.id
```

Comme vous pouvez le constater, le résultat est différent, cette fois-ci ce sont les lignes de la table LIVRES qui viennent se greffer sur la table AUTEURS.

Dans le cas d'une jointure, il est tout à fait possible de sélectionner certains attributs et pas d'autres :

À faire vous-même 22
Saisissez et testez la requête SQL suivante :

```
SELECT nom, prenom, titre
FROM AUTEURS
INNER JOIN LIVRES ON LIVRES.id_auteur = AUTEURS.id
```

À faire vous-même 23
Saisissez et testez la requête SQL suivante :

```
SELECT titre,nom, prenom
FROM LIVRES
INNER JOIN AUTEURS ON LIVRES.id_auteur = AUTEURS.id
```

Si un même nom d'attribut est présent dans les 2 tables (par exemple ici l'attribut id), il est nécessaire d'ajouter le nom de la table devant afin de pouvoir les distinguer (AUTEURS.id et LIVRES.id)

À faire vous-même 24
Saisissez et testez la requête SQL suivante :

```
SELECT titre,AUTEURS.id,nom, prenom
FROM LIVRES
INNER JOIN AUTEURS ON LIVRES.id_auteur = AUTEURS.id
```

Il est possible d'utiliser la clause WHERE dans le cas d'une jointure :

À faire vous-même 25
Saisissez et testez la requête SQL suivante :

```
SELECT titre,nom, prenom
FROM LIVRES
INNER JOIN AUTEURS ON LIVRES.id_auteur = AUTEURS.id
WHERE ann_publi>1950
```

Enfin, pour terminer avec les jointures, vous devez savoir que nous avons abordé la jointure la plus simple (INNER JOIN). Il existe des jointures plus complexes (CROSS JOIN, LEFT JOIN, RIGHT JOIN), ces autres jointures ne seront pas abordées ici.
Nous en avons terminé avec les requêtes d'interrogation, intéressons-nous maintenant aux requêtes de mise à jour (INSERT, UPDATE, DELETE).
Nous allons repartir avec une base de données qui contient une seule table :

À faire vous-même 26
Créez une nouvelle base de données que vous nommerez par exemple db_livres.db

À faire vous-même 27
Créez une table LIVRES à l'aide de la requête SQL suivante :

```
CREATE TABLE LIVRES
(id INT, titre TEXT, auteur TEXT, ann_publi INT, note INT);
```

À faire vous-même 28
Ajoutez des données à la table LIVRES à l'aide de la requête SQL suivante :

```
INSERT INTO LIVRES
(id,titre,auteur,ann_publi,note)
VALUES
(1,'1984','Orwell',1949,10),
(2,'Dune','Herbert',1965,8),
(3,'Fondation','Asimov',1951,9),
(4,'Le meilleur des mondes','Huxley',1931,7),
(5,'Fahrenheit 451','Bradbury',1953,7),
```

```
(6,'Ubik','K.Dick',1969,9),
(7,'Chroniques martiennes','Bradbury',1950,8),
(8,'La nuit des temps','Barjavel',1968,7),
(9,'Blade Runner','K.Dick',1968,8),
(10,'Les Robots','Asimov',1950,9),
(11,'La Planète des singes','Boulle',1963,8),
(12,'Ravage','Barjavel',1943,8),
(13,'Le Maître du Haut Château','K.Dick',1962,8),
(14,'Le monde des Ā','Van Vogt',1945,7),
(15,'La Fin de l'éternité','Asimov',1955,8),
(16,'De la Terre à la Lune','Verne',1865,10);
```

Nous avons déjà eu l'occasion de voir la requête permettant d'ajouter une entrée (utilisation d'INSERT)

À faire vous-même 29
Que va faire cette requête ? Vérifiez votre réponse en l'exécutant et en faisant une requête "SELECT * FROM LIVRES".

```
INSERT INTO LIVRES
(id,titre,auteur,ann_publi,note)
VALUES
(17,'Hypérion','Simmons',1989,8);
```

À faire vous-même 30
Écrivez et testez une requête permettant d'ajouter le livre de votre choix à la table LIVRES.

"UPDATE" va permettre de modifier une ou des entrées. Nous utiliserons "WHERE", comme dans le cas d'un "SELECT", pour spécifier les entrées à modifier.

Voici un exemple de modification :

À faire vous-même 31
Que va faire cette requête ? Vérifiez votre réponse en l'exécutant et en faisant une requête "SELECT * FROM LIVRES".

```
UPDATE LIVRES
SET note=7
WHERE titre = 'Hypérion'
```

À faire vous-même 32
Écrivez une requête permettant d'attribuer la note de 10 à tous les livres écrits par Asimov publiés après 1950. Testez cette requête.
"DELETE" est utilisée pour effectuer la suppression d'une (ou de plusieurs) entrée(s). Ici aussi c'est le "WHERE" qui permettra de sélectionner les entrées à supprimer.

À faire vous-même 33
Que va faire cette requête ? Vérifiez votre réponse en l'exécutant et en faisant une requête "SELECT * FROM LIVRES".

```
DELETE FROM LIVRES
WHERE titre='Hypérion'
```

À faire vous-même 34
Écrivez une requête permettant de supprimer les livres publiés avant 1945. Testez cette requête.

III) PHPMYADMIN :

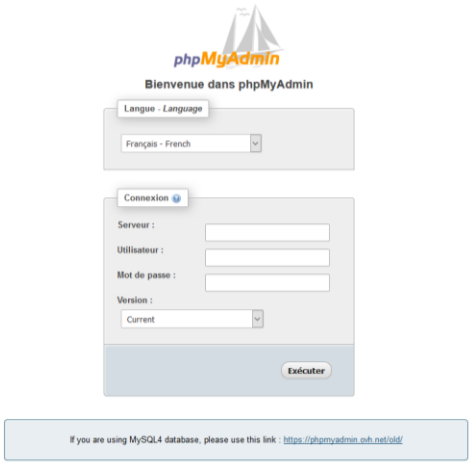
1°) Introduction :

PhpMyadmin est une interface et gestion de base de données MySQL sur un serveur PHP disponible en plusieurs langues. C'est l'une des interfaces les plus utilisées que ce soit par les hébergeurs gratuits ou payants. phpMyadmin sert donc à créer très facilement sa base de données, et y insérer des informations sans Connaissances en SQL qui est

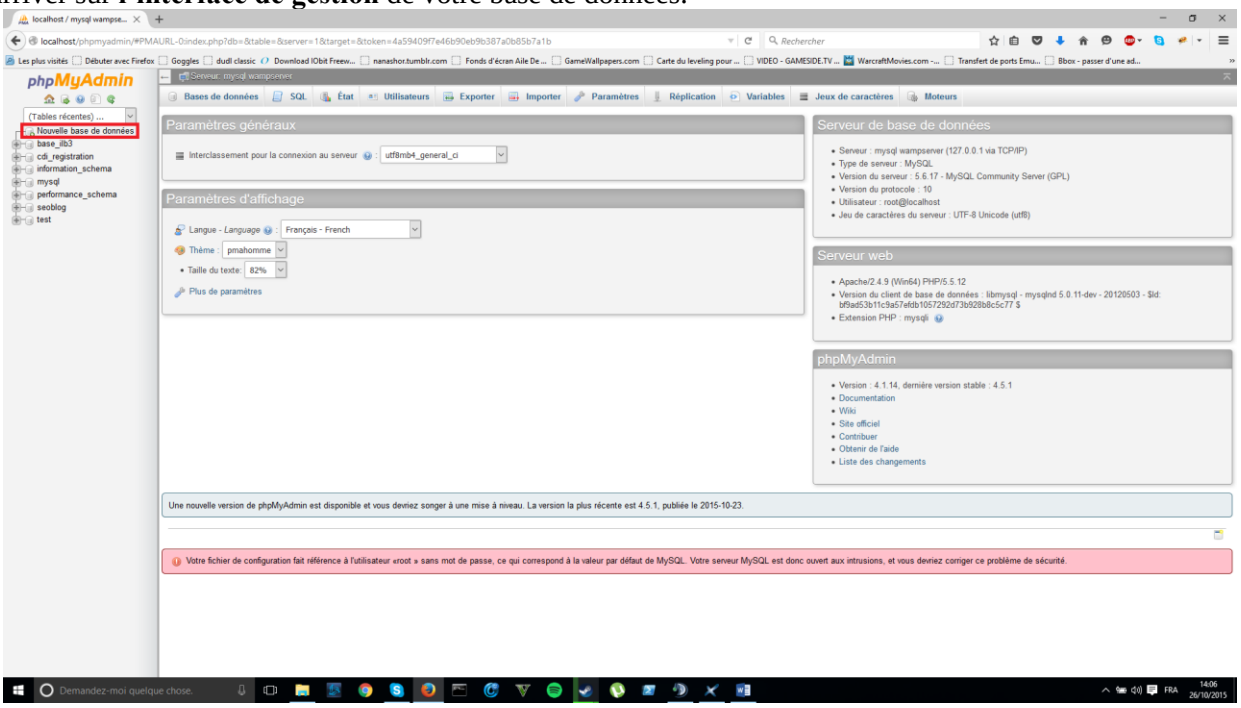
pourtant le langage incontournable de la gestion de base de données. Nous allons donc voir ensemble comment créer une base de données sur phpMyadmin et les façons différentes d'y ajouter des tables et les remplir.

2°) Connection à phpMyadmin :

Pour se connecter à phpMyadmin, le plus simple est de lancer XAMPP, de démarrer les services Apache et MySQL, et de cliquer sur Admin de la ligne MySQL. Dans Firefox, on peut aussi saisir : localhost/phpmyadmin dans la barre d'adresse. On peut aussi utiliser WAMP (ou LAMP sous Linux). Vous arriverez alors sur une interface de connexion comme celle-ci :

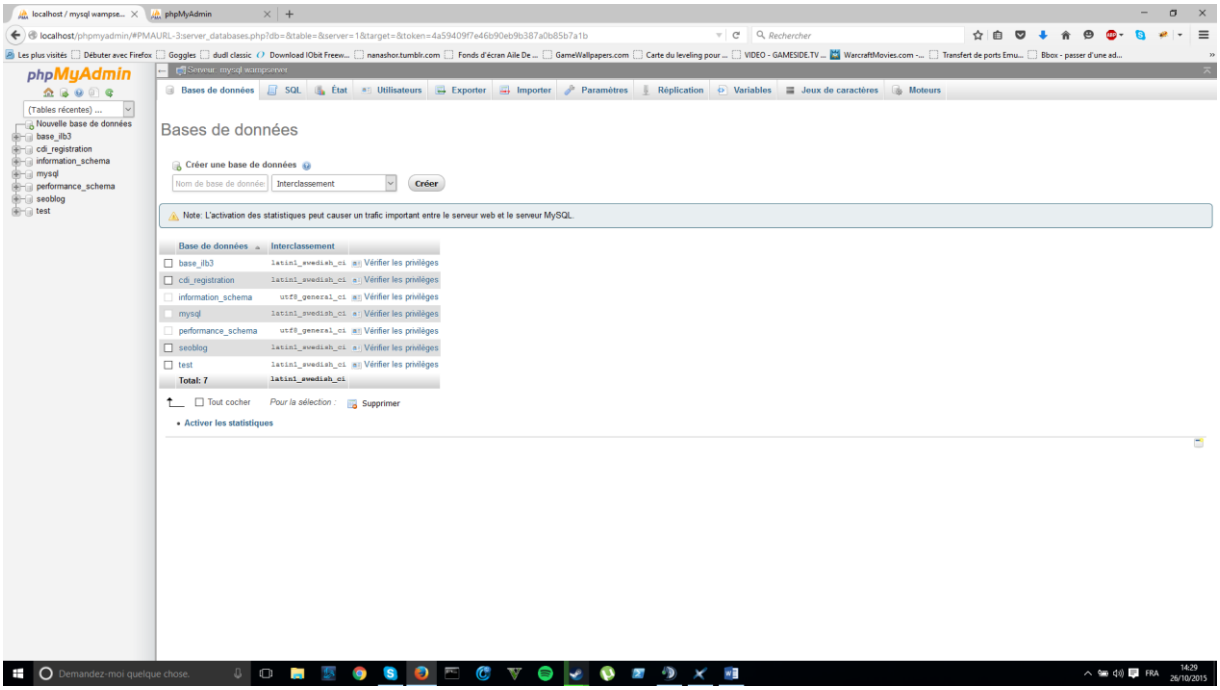


L'identifiant est root. Il n'y pas de mot de passe, il suffit d'appuyer sur entrée. Si tout se passe bien vous devriez alors enfin arriver sur **l'interface de gestion** de votre base de données.



3°) Création d'une base de données et de ses tables :

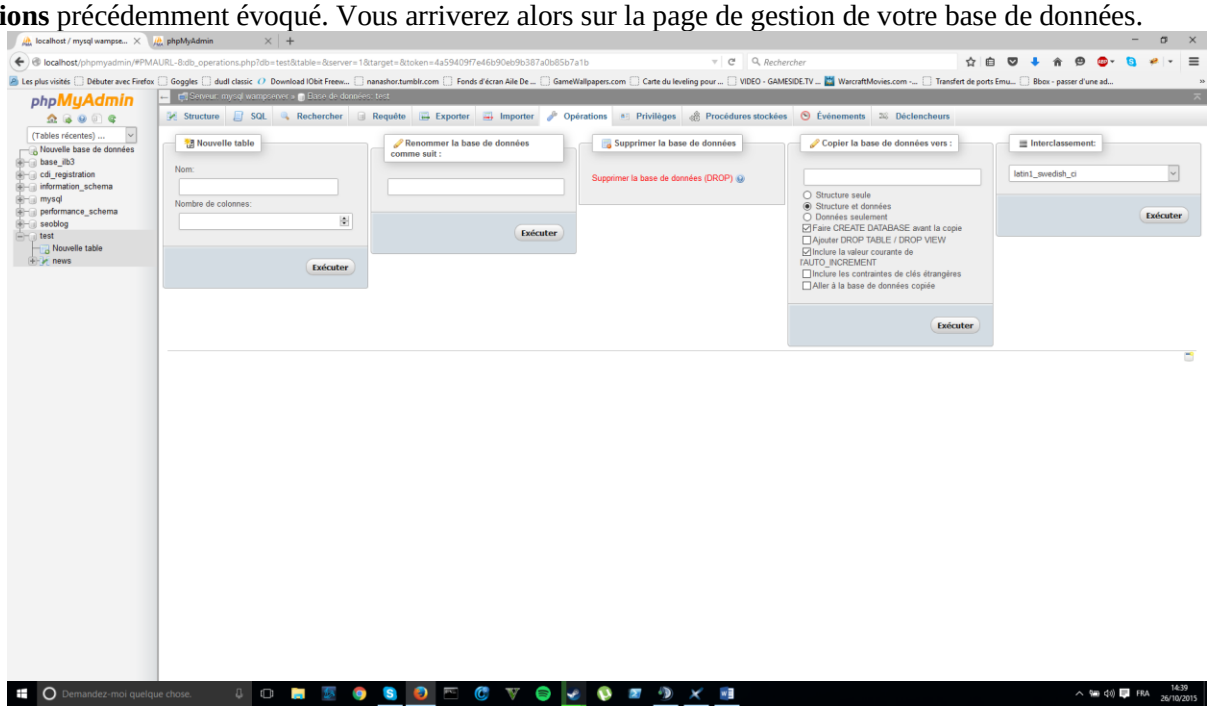
Arrivé sur votre **interface de gestion** vous allez enfin pouvoir commencer à créer votre base de données et la remplir comme bon vous semble. Pour ce faire cliquez sur **Nouvelle base de données**. Vous arriverez alors sur **l'interface de création.**



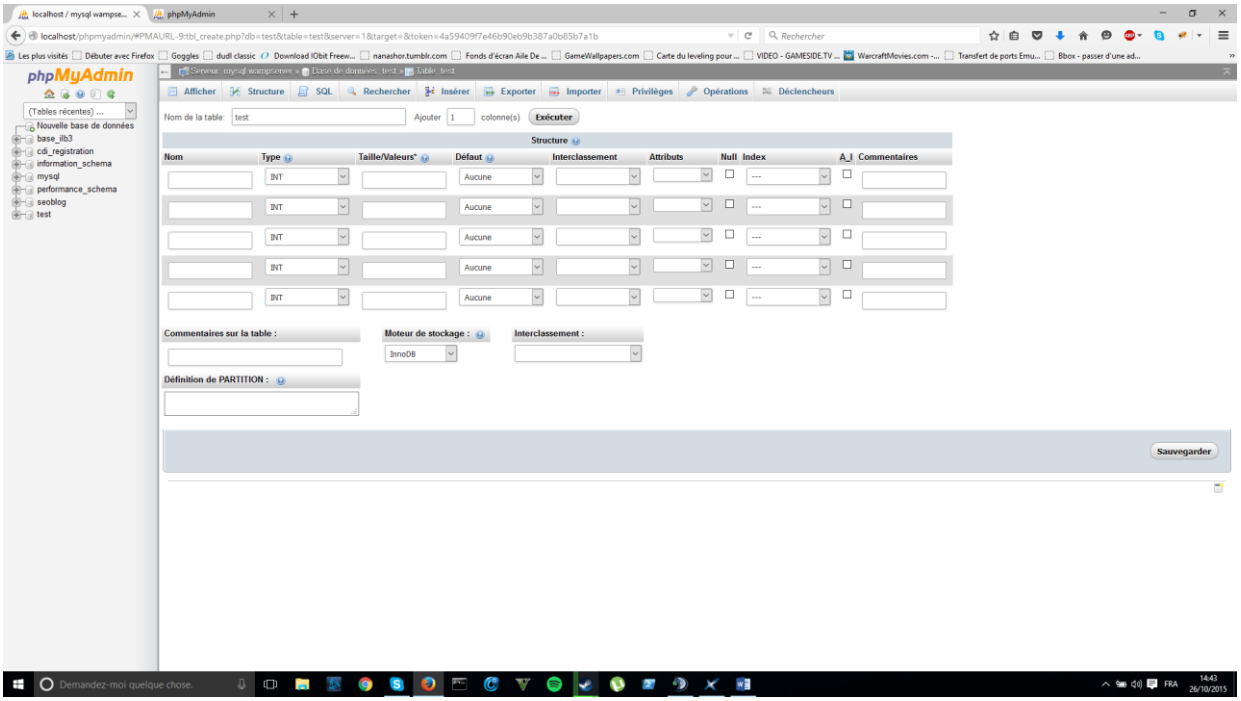
Remplissez alors le champ Nom **de base de données** avec le nom que vous voudrez lui attribuer. Choisissez l'**interclassement** qui vous convient puis cliquez sur **Créer**. Un message de confirmation apparaîtra alors. Vous avez donc créé avec succès votre base de données. Il est à noter que dans l'onglet **Opérations** de votre base de données il est possible de supprimer la base en cliquant sur **supprimer la base de données (DROP)**. Maintenant nous allons pouvoir créer nos tables. Pour cela il existe 2 possibilités :

- la créer de toute pièce.
- L'importer depuis un fichier csv.

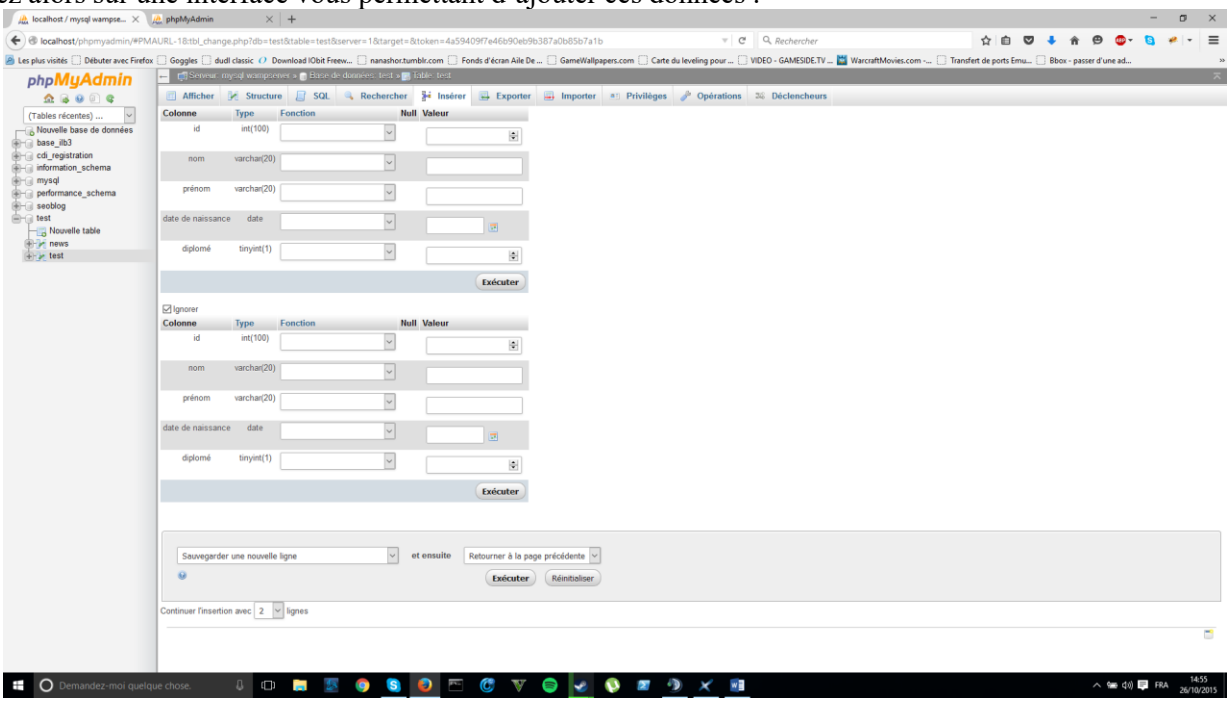
Dans le premier cas vous devrez cliquer sur la base que vous voulez modifier puis vous devrez cliquer sur l'onglet **Opérations** précédemment évoqué. Vous arriverez alors sur la page de gestion de votre base de données.



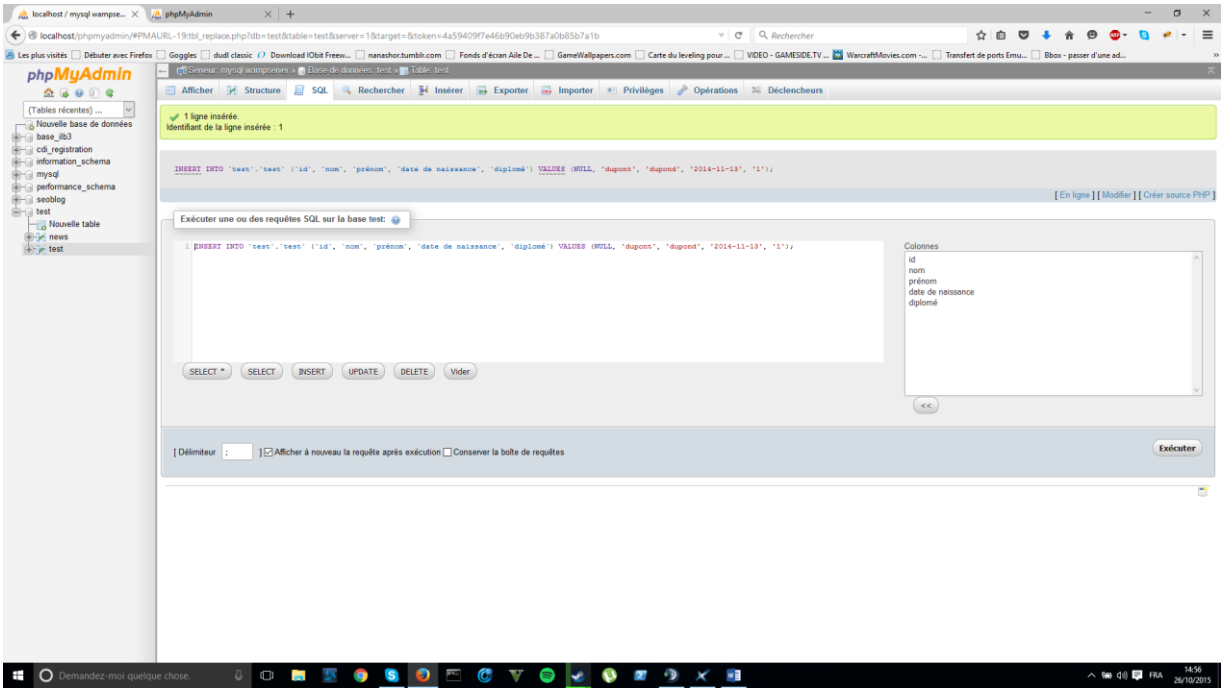
Donnez alors un **nom** à votre table ainsi que le **nombre** de colonnes que vous voulez lui attribuer puis cliquez sur **Exécuter**. Vous arriverez alors sur une interface vous permettant de configurer vos colonnes.



Remplissez alors chaque champ pour chaque colonne puis cliquez sur **sauvegarder**. La table aura donc bien été créée. Mais nous ne savons toujours pas comment implémenter des données dans nos tables me direz-vous. Et bien c'est simple il vous suffira d'aller dans l'onglet insérer après avoir cliqué sur la table que vous voulez remplir. Vous arriverez alors sur une interface vous permettant d'ajouter ces données :

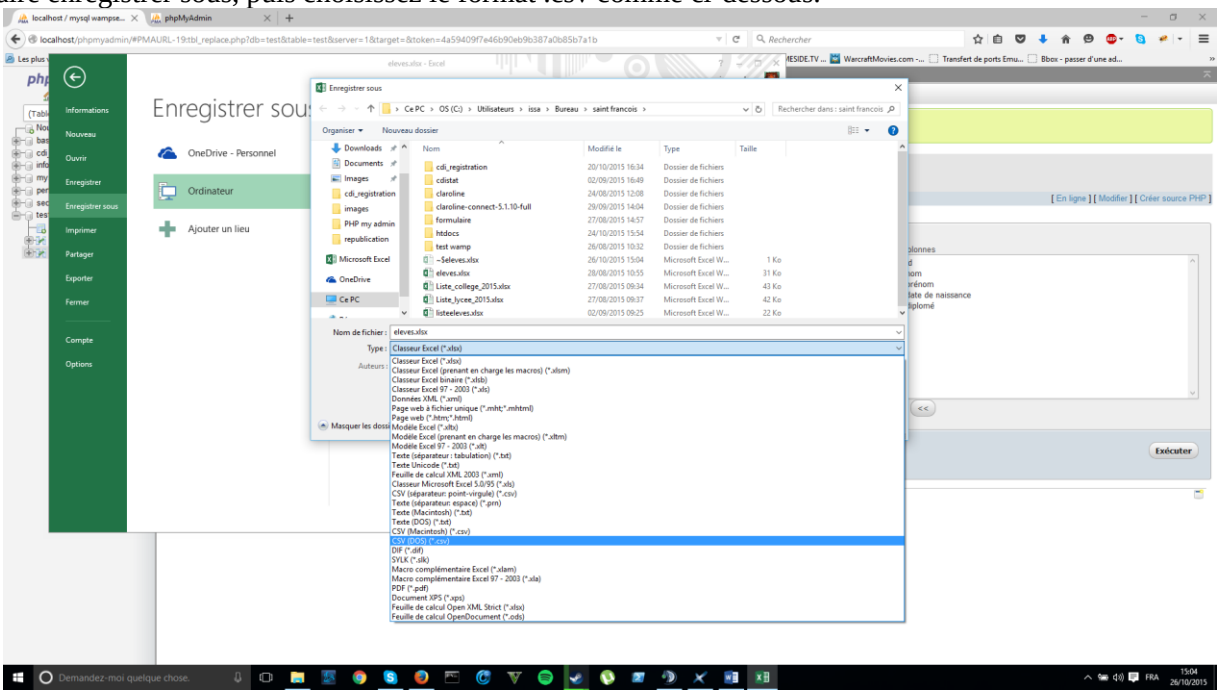


Remplissez alors ces champs disponibles avec les données correspondantes puis cliquez sur **Exécuter**. Notez qu'ils vous proposent de remplir 2 lignes à la fois. Si toutefois vous n'avez qu'une seule ligne à entrer laissez cochée la case **ignorer**. phpMyadmin vous affichera alors un message de confirmation accompagné du code qui a été exécuté comme ceci :

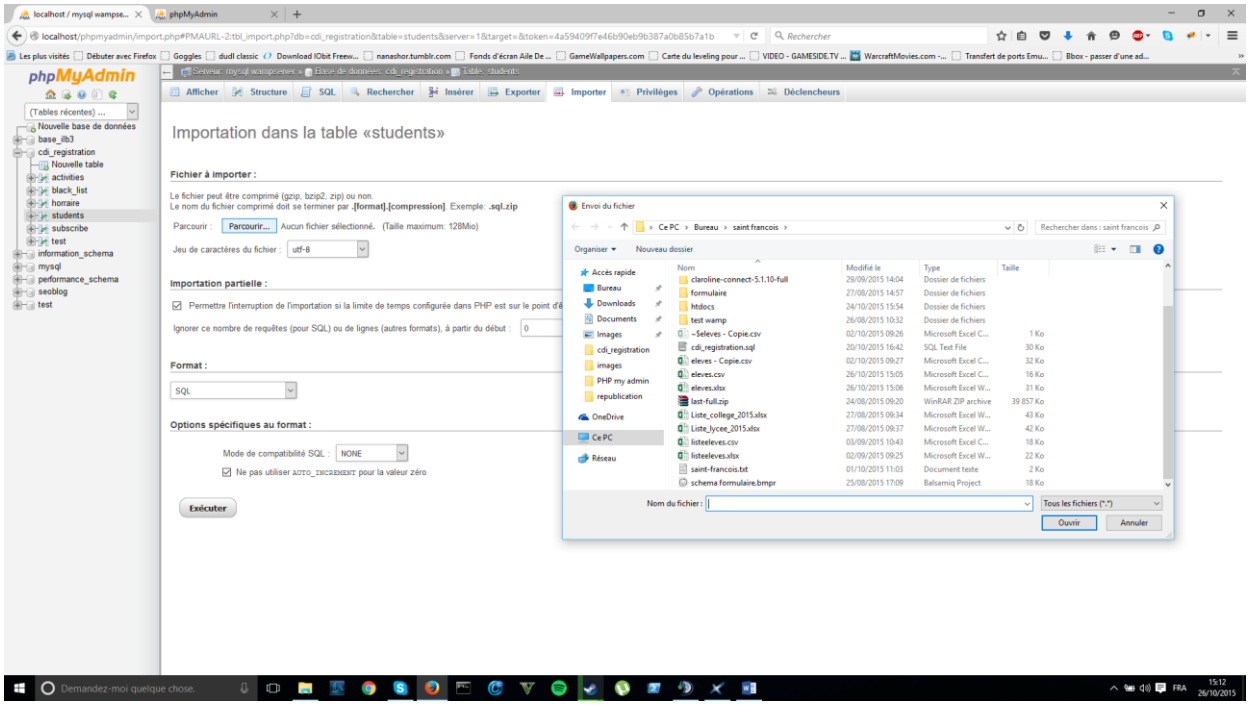


Nous avons donc vu la première façon de créer et remplir sa base de données mais il en existe une deuxième qui est bien plus pratique et efficace si vous travaillez de la bonne façon : il s'agira d'importe : il s'agira d'importer un fichier Excel au format **csv.**

Pour cela vous devrez tout d'abord comme dans le premier cas créer votre base de données et votre table. La table devra avoir exactement les mêmes colonnes que votre Excel afin d'incrémenter les bonnes données dans les bonnes colonnes. Par la suite vous devrez transformer votre fichier excel en .csv pour cela il vous faudra ouvrir votre fichier Excel faire enregistrer sous, puis choisissez le format .csv comme ci-dessous.



Cliquez ensuite sur **enregistrer** puis sur oui lorsque vous fermez votre fichier. Vous obtiendrez alors un fichier .csv compatible avec phpMyadmin. Suite à cela retournez sur phpmyadmin cliquez sur la table que vous voulez modifier puis cliquez sur l'onglet **importer**. Vous arriverez alors sur l'interface d'importation de fichier. Cliquez alors sur **parcourir** puis choisissez le fichier .csv que vous voulez importer et cliquez sur **Ouvrir**. Et enfin cliquez sur **Exécuter**.



Php vous affichera alors un message de confirmation si tout a bien fonctionné. Si vous n'avez pas réussis vérifiez que les colonnes de vos tables correspondent aux colonnes de votre excel. Note : si votre fichier Excel contient beaucoup de lignes choisissez le format **CSV using LOAD DATA**.

Remarque : phpMyAdmin permet aussi d'afficher et de créer le MCD de la base de données. Il suffit de cliquer sur « Concepteur ». Si les relations n'apparaissent pas, il faut que les bases soient créées avec InnoDB et pas avec MyISAM.
Mysql est une des bases de données les plus populaires et certainement la plus connue. Sa simplicité et sa rapide mise en production séduit beaucoup d'administrateurs. Par défaut, elle utilise le moteur MyISAM pour ses tables, et il devient vite intéressant de passer à InnoDB si l'on prévoit un usage critique de ses bases.
Pour cela, il faut aller dans le fichier de configuration mysql, (my.ini) et modifier la ligne :
default-storage_engine = InnoDB

4°) Requêtes SQL :

Pour exécuter les requêtes SQL, il suffit de se rendre sur l'onglet SQL. Le principe est alors le même qu'avec DB BROWSER FOR SQLITE.

Refaire le travail du paragraphe précédent en utilisant phpMyAdmin.

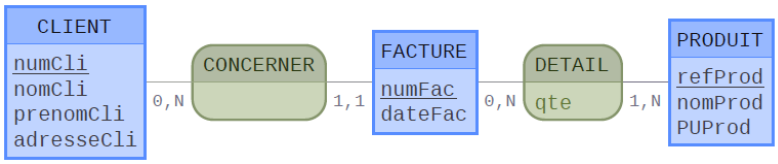
IV) LA CONSOLE MYSQL :

Sous XAMPP, elle s'ouvre de la manière suivante :
Cliquer sur Shell, une console s'ouvre, et saisir la ligne : mysql -h localhost -u root.

Elle s'ouvre aussi à partir de WAMP (console MySQL). Tout ce qui a été fait précédemment peut être fait directement dans la console MySQL, en utilisant les lignes de commandes correspondant aux requêtes SQL.

Plutôt que de taper chaque requête SQl, on peut les mettre dans un fichier texte sous notepad par exemple, au format.sql, et les exécuter toutes d'un coup avec la commande : SOURCE nom_fichier.sql.

Travail à faire :
L'entreprise QuiVendTout a créé une base de données lui permettant d'éditer les factures de leurs clients. Le MCD et le MLD de cette base de données vous sont donnés ci-dessous :



Client (numCli, nomCli, prenomCli, adresseCli)
Facture (numFac, dateFac, #numCli)
Detail (#numFac, #refProd, qte)
Produit (refProd, nomProd, PUProd)

1°) Création de la base de données :

Lancer XAMPP. Ouvrir la console MySQL.
Créer la base avec la commande :

```
create database facture set 'utf8';
```

Se placer sur cette base :

```
use facture;
```

Sous XAMPP : faire un copier-coller des trois fichiers : creer_facture.sql, jeu_essai_facture.sql et detruire_facture.sql dans le dossier : C:\xampp\mysql\data\facture.

Sous WAMP : faire un copier-coller des trois fichiers : creer_facture.sql, jeu_essai_facture.sql et detruire_facture.sql dans le dossier : C:\wamp64\bin\mysql\mysql***\data\facture.

Les scripts de création et le jeu d'essai se trouvent dans les fichiers creer_facture.sql et jeu_essai_facture.sql.
Pour exécuter ces scripts il faut taper les commandes suivantes (attention, il est possible qu'il faille ajouter le chemin complet au nom du fichier correspondant : (c:\xampp\mysql\data\facture\).

```
source creer_facture.sql
source jeu_essai_facture.sql
```

Si tout s'est bien passé lorsque vous tapez la commande :

```
show tables ;
```

vous devez obtenir :

```
+--------------------+
| CLIENT             |
| DETAIL             |
| FACTURE            |
| PRODUIT            |
+--------------------+
```

Vous pouvez consulter la structure des tables grâce à la commande desc. Par exemple :

```
desc CLIENT;
```

Enfin vous pouvez consulter le contenu d'une table en tapant la commande :

```
select * from CLIENT;
```

par exemple.
Voici le contenu de la base de données après insertion du jeu d'essai :

```
|                    CLIENT                      | |            FACTURE            |
+---------+------------+-----------+------------+ +---------+------------+---------+
| numCli  | nomCli     | prenomCli | adresseCli | | numFac  | dateFac    | numCli  |
+---------+------------+-----------+------------+ +---------+------------+---------+
|       1 | BANAZIAI   | Jules     | Grenoble   | |       1 | 2019-06-14 |       2 |
|       2 | DONGEPLI   | Armelle   | Limoges    | |       2 | 2019-12-26 |       2 |
|       3 | BOBAINO    | Julie     | Nîmes      | |       3 | 2019-03-14 |       1 |
|       4 | BOUTIDICHT | Maxime    | Grenoble   | +---------+------------+---------+
+---------+------------+-----------+------------+
|          DETAIL           | |                   PRODUIT                    |
+---------+---------+-------+ +---------+------------------------+---------+
| numFac  | refProd | qte   | | refProd | nomProd                | puProd  |
+---------+---------+-------+ +---------+------------------------+---------+
|       1 |       2 |     8 | |       1 | Sucre                  |    1.20 |
|       1 |       3 |     2 | |       2 | Céréales               |    1.30 |
|       2 |       6 |     7 | |       3 | Biscottes              |    1.15 |
|       3 |       1 |     2 | |       4 | Poudre petit déjeuner  |    2.30 |
|       3 |       3 |     2 | |       5 | Café                   |    2.50 |
|       3 |       5 |     9 | |       6 | Thé                    |    3.10 |
+---------+---------+-------+ +---------+------------------------+---------+
```

2°) Mise à jour d'une base de données et contraintes d'intégrité :

a) Le script detruire_facture.sql censé écraser la base de données des factures provoque des erreurs. Pourquoi ? Corrigez le pour qu'il fonctionne.

b) Après avoir recréé la base de données, donnez les lignes à insérer dans les tables qui permettraient d'éditer la facture suivante :

```
                          FACTURE 4
Numéro client: 5                      date: 3/12/2019
Lea RICCOH
Orléans
Ref   Nom           Qte     PU    Total
3     Biscottes       2   1.15     2.30
6     Thé             5   3.10    15.50
7     Jus d'Orange    2   2.40     4.80
                                   -----
                                   22.60
```

c) Essayez d'effacer la Poudre petit déjeuner puis le sucre. Que se passe-t-il ? Pourquoi ?

d) Essayez d'insérer la ligne (1,3,12) dans la table DETAIL. Que se passe-t-il ? Pourquoi ?

e) Constant Mépatro, votre gestionnaire de stock, vous demande de changer la référence du Thé qui doit maintenant être 12342. Tapez la commande qui permet de le faire.
Que se passe-t-il ? Pourquoi ?

f) Les ventes de l'entreprise sont en baisse, pour remédier à ce problème le patron décide de baisser tous les prix de 10 %. Tapez la commande qui permet de le faire.

V) <u>BASES DE DONNEES SOUS PYTHON</u> :

Le travail qui suit se fait avec la base de données précédente : facture.
Il nécessite également qu'un serveur SQL soit présent et lancé sur le poste de travail. Il suffit donc de lancer XAMPP (ou WAMP (ou LAMP sous Linux)) si ce n'est pas déjà fait.
<u>Rappel</u> : par défaut le login de ce serveur SQL est root, et pour le mot de passe il suffit d'appuyer sur entrée (mot de passe='').

Le fichier pythonSQL.py contient un exemple permettant de se connecter à une base de données MySQL et de faire les opérations suivantes :

- Ajouter de nouveau produits dans la table PRODUIT.
- Consulter la liste des clients habitant une certaine ville.
- Afficher les factures à l'écran.

Pour faire fonctionner ce programme il faut avoir installer le connecteur MySQL : pip install mysql-connector-python (et sqlalchemy).

Les exemples reposent sur trois objets principaux :

- engine qui est l'objet gérant les interactions avec le serveur de BD.
- Connection qui est l'objet gérant la connexion au serveur de BD et qui permet de lancer des requêtes.
- Le résultat des requêtes de consultation qui est un objet bizarre mais sur lequel on peut itérer.

En observant le code des trois fonctions déjà implémentées, écrire les fonctions suivantes :
a) Une fonction permettant d'ajouter de nouveaux clients.
b) Une fonction permettant d'afficher la liste des produits compris entre dans une fourchette de prix (les bornes de la fourchette étant demandées à l'utilisateur).
c) Une fonction permettant d'effacer un client en fonction de son nom et son prénom (si le client ne peut pas être effacé, l'indiquer à l'utilisateur).
d) Une fonction permettant de faire afficher le numéro de la facture avec le nom et le prénom du client, la date, les produits achetés et leurs quantités.